

NAME

sumtag - stamp files with an XXH3 checksum and metadata for corruption and duplicate detection

SYNOPSIS

sumtag [*OPTIONS*] *DIRECTORY* [*DIRECTORY*...]

DESCRIPTION

sumtag recursively scans each *DIRECTORY*, computes an XXH3 checksum of every file's contents, and stores the checksum together with metadata in an extended attribute named **user.sumtag** on the file. The recorded metadata makes it possible to later detect silent data corruption and to find duplicate files. Those higher-level analyses are left to other tools, with one exception built in here: the single-pass integrity check offered by **--verify**.

At least one *DIRECTORY* is required; there is no current-directory default. Any operation that scans a directory requires an explicit directory argument, so that a recursive operation is never aimed at the working directory by omission. Pass **.** explicitly to scan the current directory.

For each file, sumtag reads any existing **user.sumtag** attribute and compares the recorded modification time to the file's current mtime. If they match, the file is considered up to date and is skipped. Otherwise the file's contents are hashed and the attribute is written.

Within each directory, files are processed in ascending alphabetical order and subdirectories are then recursed into in the same order, so the processing order can be followed against an **ls(1)** listing. This holds in every mode.

RE-HASHING

A file is hashed, or re-hashed, when any of the following hold:

- ☉ the **--force** option was given;
- ☉ the **user.sumtag** attribute is absent or unreadable;
- ☉ the stored modification time is older than the file's current mtime;
- ☉ the stored schema version has a lower major number than the running software.

Freshness is algorithm-agnostic: a file with any current digest, regardless of which algorithm produced it, counts as up to date. A stored digest is never recomputed merely because a different algorithm is now preferred; when a re-hash does happen for one of the reasons above, it replaces the file's single stored digest rather than keeping the old one alongside it.

OPTIONS

-n, --dry-run

Scan and report what would be done, but write nothing -- no extended attributes and no database rows. Output matches a real run. Cannot be combined with **--force**.

-q, --quiet

Suppress normal output. Repeat (**-qq**) to also suppress error messages on standard error. Mutually exclusive with **--verbose**.

-v, --verbose

Explain the decision made for each file, including skips. Without this option, each file touched is announced by its bare path alone. Repeat (**-vv**) for deep internal detail intended for debugging. Mutually exclusive with **--quiet**.

--progress

Show a live within-file progress indicator for large files. If it appears together with **-q**, whichever option appears last on the command line wins, and a warning is printed to standard error. The indicator's line shows the file's size, the current hashing rate, a bar, the percentage complete, elapsed time, and an ETA, e.g.:

```
99.3GiB 40.1MiB/s [=====> ] 84% 0:05:12 ETA 58s
```

The bar is the only field whose width varies: it absorbs whatever room the fixed-width fields leave, and resizes live when the terminal window is resized.

--si Display **--progress** sizes and rates using decimal (SI, powers-of-1000: **kB/MB/GB**) units instead of the default binary (powers-of-1024: **KiB/MiB/GiB**) units.

-f, --force

Re-hash every file unconditionally, ignoring any existing metadata. Cannot be combined with **--dry-run** or **--verify**. Combined with **--import** or **--locate**, overrides their default refusal to compute (see below). This option does *not* override **@sumtag-ignore** markers; use **--no-ignore** for that.

--database VALUE

Name a database to act on. *VALUE* is a SQLite file path, or a scheme:// DSN. A value matching the pattern `^[a-z][a-z0-9+-.]*://` is treated as a DSN (**mysql://**, **postgresql://**); any other value, including one that merely contains a colon, is a SQLite file path. Only SQLite is implemented; a DSN is recognised and rejected as not yet supported. Database paths are stored relative to the file's mount point so they survive remounting at a different location. **--database** takes no action by itself; it requires at least one of **--sum**, **--import**, or **--locate** to say what to do with it.

--sum

Re-hash each file per the normal mtime-based decision (see **RE-HASHING**) and mirror the result, in addition to the extended attribute, into the database named by **--database**, which this option requires.

--import

Do not compute any checksum by default; only copy metadata already present in each file's extended attribute into the database. Requires **--database**. Files lacking usable metadata are skipped and reported. Useful for populating a database from an archive that was hashed on a previous run. Combined with **--force**, computes and mirrors every file instead of only propagating what already exists. Combined with **--sum**, is redundant (the normal decision already computes and mirrors) but not an error. Cannot be combined with **--verify**.

--locate

Stat every file visited and write filesystem metadata (size, permissions, ownership, link count, device, and timestamps) to the database, regardless of whether xattr work was done. Implies **--import**: existing extended-attribute metadata is propagated as well, so **--locate** alone does everything **--import** alone would, plus stat columns. Requires **--database**. Combined with **--force**, computes checksums for every file too, the same override **--import** accepts. Useful as a periodic filesystem inventory pass (analogous to **updatedb**).

--verify

Read-only integrity check: recompute each file's checksum and compare it to the stored value, reporting any disagreement. Nothing is written, even on a mismatch. A mismatch while the stored modification time still matches the file's mtime indicates silent corruption; a mismatch accompanied by a newer mtime indicates an ordinary modification whose stamp is merely stale. Files with no usable metadata are reported as unverifiable. Cannot be combined with **--database** (nor **--sum**, **--import**, or **--locate**, which all require it), or **--force**; combining with **--dry-run** is a redundant no-op and is permitted. See **EXIT STATUS** below.

--remove

Remove the **user.sumtag** xattr from every file in the tree. A testing/reset utility, not a data-integrity primitive: no mtime comparison, no hashing, just an unconditional delete of whatever attribute happens to be present. Files with no attribute are left alone. Combined with **--dry-run**, previews which files would lose their stamp without removing anything. Cannot be combined with **--database** (nor **--sum**, **--import**, or **--locate**, which all require it), **--verify**, or **--force**.

--prescan

Walk the tree once up front to count the files that will be checksummed and their total size, then prefix each hash/verify announcement with an *nnn/mmm* file counter and a bytes-so-far/total

counter, e.g.:

042/137 118.2MiB/4.2GiB hash /backup/vault/photo0042.dng (reason)

The counted set mirrors whichever mode is running: for the normal hash/stamp pass it is the files the re-hash decision (or **--force**) will actually hash; for **--verify** it is every file with a usable stored digest. During the counting walk each directory visited is announced before any file metadata inside it is read (the bare directory path, or *prescan path* with **-v**; suppressed by **-q**), so the otherwise-silent up-front pass shows its own progress on a large tree. The prescan pass is a prediction from a separate walk, not a guarantee -- counts can drift if the tree changes between the two passes. Cannot be combined with **--remove**.

--no-ignore

Disregard all **@sumtag-ignore** marker files for this run, processing every directory regardless of markers.

--version

Print version information and exit.

-h, --help

Print a usage summary and exit.

IGNORE MARKERS

A directory containing a file named **@sumtag-ignore** is exempted along with its entire subtree: sumtag does not descend into it, and nothing beneath it is read, hashed, stamped, or mirrored to a database. The marker file itself is never hashed. Only the file's presence matters; its contents are ignored and reserved for future use. The exemption applies in every mode and takes precedence over **--force**; use **--no-ignore** to override it. A marker found at a directory named directly on the command line is honoured, with a warning printed to standard error.

EXIT STATUS

Without **--verify**, sumtag exits **0** on success and non-zero if an error prevented completion.

With **--verify**:

- 0** all checked files verified intact.
- 1** one or more checksum mismatches (corruption) were found.
- 2** unreadable files or other errors prevented a complete check.

FILES

user.sumtag

Extended attribute holding each file's checksum and metadata as UTF-8 JSON.

@sumtag-ignore

Marker file that exempts its directory and subtree (see IGNORE MARKERS).

EXAMPLES

Scan the current directory:

```
sumtag .
```

Scan two archives:

```
sumtag /data /backup
```

Preview without writing anything:

```
sumtag -n /data
```

Stamp and also mirror into a SQLite database:

```
sumtag --database /var/lib/sumtag.sqlite --sum /data
```

Import existing metadata into a database without re-reading files:

```
sumtag --database /var/lib/sumtag.sqlite --import /data
```

Take a filesystem inventory pass, capturing stat metadata alongside any existing checksums:

```
sumtag --database /var/lib/sumtag.sqlite --locate /data
```

Verify an archive against its stored checksums:

```
sumtag --verify /backup
```

NOTES

Timestamps are recorded in UTC with microsecond precision. Mtime comparisons truncate both the stored and live values to the same precision, so the comparison is symmetric.

SEE ALSO

xattr(1), getfattr(1), setfattr(1), cksum(1)

AUTHOR

The sumtag authors.